

Data Handling: Import, Cleaning and Visualisation

Exercise/Workshop 1: Tools and functions

Dr. Aurélien Sallin
(University of St. Gallen)

Exercises

Exercise A: Setting up a Working Environment

Setting Up a Basic Folder Structure and Working Environment in RStudio

Follow these steps to set up a structured and organized environment for your R project in RStudio:

1. Create a New Folder on Your Laptop:
 - Navigate to a location of your choice on your personal laptop (e.g., your Documents folder).
 - Right-click and select New Folder to create a new folder.
 - Name this new folder `r_course`.
2. Create Subfolders for Data and Code:
 - Open the newly created `r_course` folder.
 - Right-click inside the folder and select New Folder again to create two subfolders:
 - Name the first subfolder `data`.
 - Name the second subfolder `code`.
3. Create an RStudio Project in the `r_course` Folder:
 - Open RStudio.
 - Click on the Project button in the top-right corner of the RStudio window.
 - Select New Project... from the drop-down menu.
 - In the pop-up window, choose Existing Directory.
 - Click Browse... and navigate to the `r_course` folder you created in Step 1.
 - Select the `r_course` folder and click Create Project.
4. Confirm Your Working Directory:
 - When the project is created, RStudio will open it and set the working directory to the `r_course` folder automatically.
 - To check the current working directory, run the following command in the Console:

Exercise B: R Scripts

1. Switch to the R console and type the following line of code and hit enter (see example from above).

```
print("Hello world")
```

You should see the words "Hello world" printed on screen. This is the usual way of working with R in an interactive session. However, in most circumstances, it makes sense to write the R code to an R script (in order to store and document it) and then execute the code from there.

2. In the RStudio Menu-bar select **File/New File/R Script** to create a new file, shown/opened in the Script pane.
3. Type `print("Hello world")` to the first line of the script, and click on 'Run' to execute the code in the console.
4. Save the file as `hello_world.R` (File/Save As...) in the sub folder `code` (created in the previous exercise).
5. Type the following command into the command line and hit enter:

```
source("code/hello_world.R")
```

```
source("../r_course/code/hello_world.R")
```

Now you know how to execute R code directly in the console (interactive session), how to execute lines of code written to an R script, as well as how to execute the entire R script (stored on disk) from the command line.

Exercise C: Saving output in a file

In Exercise B, you printed the phrase “Hello world” to your console. Now, you will save the phrase to a text file.

1. To make sure you are in the right folder, you can type `getwd()` into the console; it will show your current working directory. You should see `{some path to folder}/r_course` as an output.
2. Now, let's save the output "Hello world" to a file named “hello_world_file.txt”. Type the following command into the console:

```
cat("Hello world", file = "hello_world_file.txt")
```

4. In the file pane, open the folder `../r_course/code`. Make sure it contains a file named “hello_world_file.txt”. You can open it to verify its content.

Exercise D: Write a Sum Function

In the lecture on Programming with Data, we have discussed the most basic programming concepts in R. Among other things, you have learned how we can use for-loops to sum up numbers stored in a numeric vector.

1. Use the code example in the respective lecture notes and the following empty function construct to implement a function that takes a numeric vector as input and returns the sum of the vector's elements.

```
my_sum <- function(x){
  }

```

2. Define the vector `numbers = c(1, 2, 3)`. Then, compare your function `my_sum()` with the `sum()` function (the pre-implemented sum function in R) when using `numbers` as the input argument: Do you get the same output for `my_sum(numbers)` and `sum(numbers)`?

Exercise E: Loop over fast and slow sloth (Supplementary exercise)

We can use loops to simulate natural processes over time. Write a program that calculates the populations of two kinds of sloths over time. At the beginning of year 1, there are 1000 slow sloths and 1 fast sloth. This one fast sloth is a new mutation that is genetically able to use roller blades. Not surprisingly, being fast gives it an advantage, as it can better escape from predators. Each year, each sloth has one offspring. There are no further mutations, so slow sloths beget slow sloths, and fast sloths beget fast sloths. Also, each year 40% of all slow sloths die each year, while only 30% of the fast sloths do. So, at the beginning of year one there are

1000 slow sloths. Another 1000 sloths are born. But, 40% of these 2000 slow sloths die, leaving a total of 1200 at the end of year one. Meanwhile, in the same year, we begin with 1 fast sloth, 1 more is born, and 30% of these die, leaving 1.4.

Enter the first year in which the fast sloths outnumber the slow sloths.

```
# Initialize populations for Year 1
slow_sloths <- 1000
fast_sloths <- 1

# Set the rates of death
slow_death_rate <- 0.40
fast_death_rate <- 0.30

# Initialize year counter
year <- 1

# Loop until fast sloths outnumber slow sloths
while (slow_sloths >= fast_sloths) {
  # Calculate the new population after births and deaths for slow sloths
  slow_sloths <- slow_sloths * 2 * (1 - slow_death_rate)

  # Calculate the new population after births and deaths for fast sloths
  fast_sloths <- fast_sloths * 2 * (1 - fast_death_rate)

  # Increment the year
  year <- year + 1
}

# Print the first year in which fast sloths outnumber slow sloths
print(year)

# Print
cat("The fast sloths outnumber the slow sloths in year:", year , "\n")
```